VisionInspectAI-

Manufacturing Defect Detection & Quality Inspection System

Milestone 4 Report

Prepared By: Yash Goyal
Team: Team-2

Table of Contents

1. Milestone 4 Overview

Milestone 4 focused on **analyzing and defining the deployment approach for VisionInspect AI**.

By the beginning of this milestone, the project had already progressed through dataset preparation, AI model development, model evaluation, and integration of the AI pipeline with the Flask backend and React frontend. Therefore, the primary requirement in Milestone 4 was to determine **how the completed application should be deployed** in a portable and publicly accessible environment.

The work performed in this milestone primarily involved:

- Analyzing deployment requirements.
- Selecting the deployment technology stack.
- Designing the deployment architecture.
- Deciding the containerization strategy.
- Deciding how the large PatchCore model would be hosted.
- Defining configuration management through environment variables.
- Defining the public-access mechanism.
- Designing the end-to-end deployment workflow.
- Preparing the deployment methodology and documentation.
- Handing over the finalized deployment plan to **Himabindhu Ravuri for implementation**.

Thus, my role in Milestone 4 was primarily deployment planning, technical decision-making, architecture design, and documentation rather than the actual deployment implementation.

2. Brief Summary of Previous Milestones

Milestone 4 was based on the work completed during the first three milestones.

Milestone 1 – AI Foundation

Milestone 1 focused on the MVTec AD dataset and the initial AI pipeline.

Major activities included:

- Dataset exploration.
- Dataset validation.
- Data preprocessing.

- Image resizing and organization.
- Initial PatchCore anomaly detection development.

Milestone 2 – Model Development and Evaluation

Milestone 2 expanded the AI system into a two-stage inspection pipeline consisting of:

- PatchCore anomaly detection.
- EfficientNet-B0 defect classification.
- Classification dataset preparation.
- Model training.
- Unified prediction pipeline.
- Model evaluation.

The resulting workflow was:

Input Image → PatchCore → Normal/Defective → EfficientNet-B0 → Defect Type

PatchCore achieved 95.84% anomaly-detection accuracy, while EfficientNet-B0 achieved 80.16% defect-classification accuracy.

Milestone 3 – AI and Web Application Integration

Milestone 3 focused on integrating the AI system with the web application.

Major activities included:

- Flask backend integration.
- React frontend integration.
- Image upload.
- AI prediction.
- Anomaly heatmap generation.
- Quality assessment.
- Inspection interface enhancement.
- Reports integration.
- Inspection history and filtering.

By the end of Milestone 3, the application supported an end-to-end inspection flow from image upload to AI prediction and result visualization. This integrated application became the basis for the deployment planning carried out in Milestone 4.

3. Objectives of Milestone 4

The main objective of Milestone 4 was to **define a suitable deployment strategy for the completed VisionInspect AI application**.

The specific objectives were:

1. Analyze the deployment requirements of the existing application.
2. Identify suitable deployment technologies.
3. Decide the frontend and backend containerization approach.
4. Select a container orchestration mechanism.
5. Decide how the large PatchCore model checkpoints should be hosted.
6. Define a configuration strategy using environment variables.
7. Select a production web server for the React application.
8. Decide how the application should be made publicly accessible.
9. Design the deployment architecture.
10. Define the end-to-end deployment workflow.
11. Prepare technical documentation for implementation.
12. Hand over the finalized deployment methodology to the implementation member.

4. Deployment Requirements Identified

Before deciding the deployment methodology, the requirements of the existing application were analyzed.

4.1 Multiple Application Components

The application contains:

- React frontend.
- Flask backend.
- AI inference components.

Therefore, a deployment approach was required that could keep the frontend and backend logically separate while allowing them to communicate.

4.2 Large AI Model

The PatchCore model checkpoints are approximately **3.75 GB**.

Therefore, including the model directly in the Git repository or Docker image was considered unsuitable because it would significantly increase the size of the deployment artifacts.

4.3 Environment-Specific Configuration

The frontend communicates with the backend through API URLs, which may differ between development and deployment environments.

Therefore, API URLs should not be hardcoded into the application.

4.4 Frontend Routing

The React application uses client-side routing. Therefore, the selected production web server needed to support proper serving of the React build and client-side routes.

4.5 Public Accessibility

The application needed to be accessible remotely for demonstration and evaluation.

Therefore, a mechanism for exposing the application publicly was required without relying on traditional cloud hosting.

5. Proposed Deployment Methodology

Based on the identified requirements, the following deployment methodology was proposed and finalized for implementation.

1. Containerize Frontend and Backend

Use separate Docker containers for:

- React frontend.
- Flask backend.

This provides separation between the two application components.

2. Use Docker Compose

Use Docker Compose to manage the frontend and backend services together.

3. Externalize Configuration

Use environment variables for:

- API URLs.
- Ports.
- Deployment-specific configuration.

This allows the same application to be used across different environments without modifying the source code.

4. Host PatchCore Externally

Host the approximately 3.75 GB PatchCore checkpoints on **Hugging Face Hub** rather than including them directly in the repository or Docker image.

The proposed approach was to retrieve the model automatically when the backend container starts.

5. Use Nginx for Frontend Serving

Use Nginx inside the frontend container to serve the production React build and support client-side routing.

6. Use Cloudflare Tunnel

Use Cloudflare Tunnel to make the application accessible through public URLs.

7. Define Separate Frontend and Backend Access

The proposed service mapping was:

- Frontend → port **3000**
- Backend → port **5000**

8. Define Validation Requirements

The deployment was planned to be validated through the complete application workflow:

Login → Image Upload → AI Inspection → Results → Dashboard

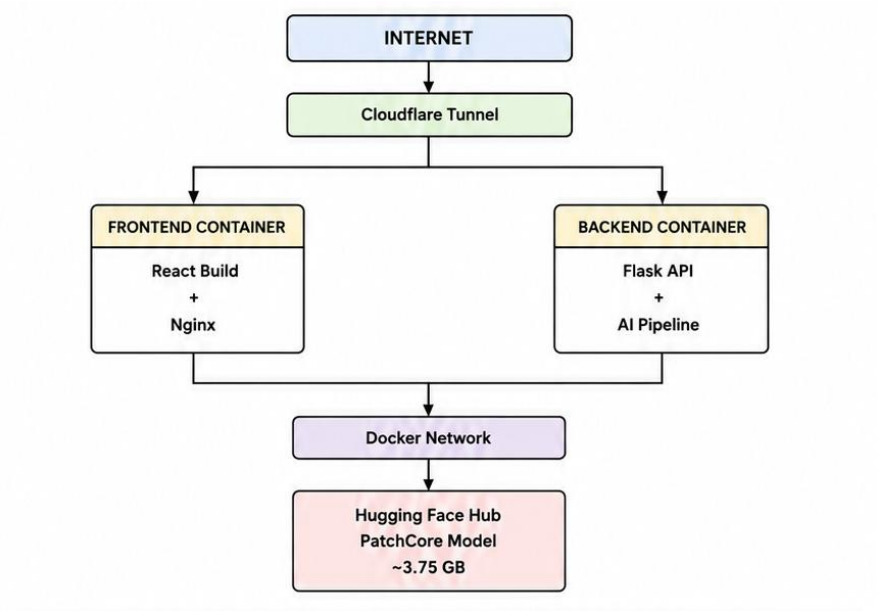
6. Selected Technology Stack

Layer	Selected Technology	Planned Purpose
Containerization	Docker	Containerize application services
Orchestration	Docker Compose	Manage frontend and backend services
Web Server	Nginx	Serve React production build
Model Hosting	Hugging Face Hub	Host PatchCore checkpoints
Public Exposure	Cloudflare Tunnel	Provide public access
Configuration	Environment Variables	Manage API URLs and configuration
Version Control	Git & GitHub	Manage source code and configuration

The technology stack was selected based on the deployment requirements identified for the existing application.

7. Proposed Deployment Architecture

The proposed architecture separates the application into frontend and backend containers.



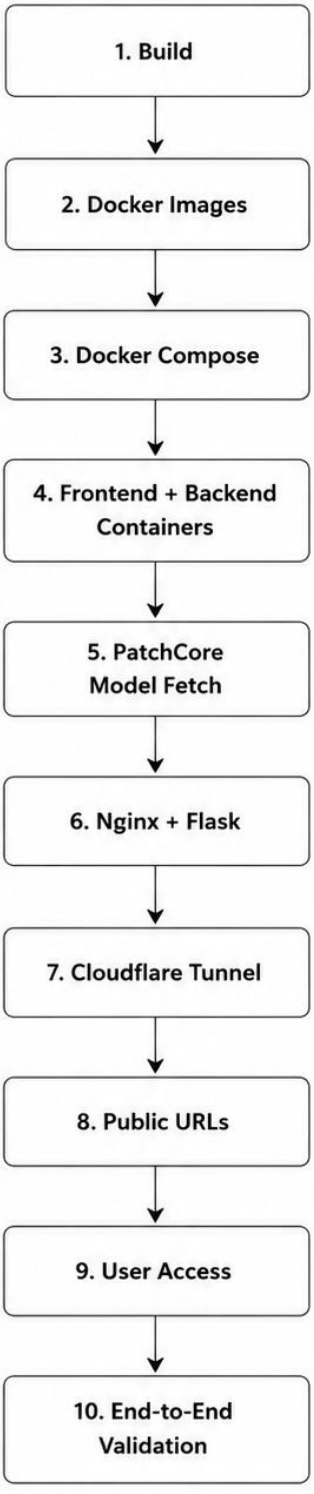
Supporting Components

- Docker Network for communication between services.
- Hugging Face Hub for PatchCore model storage.
- Cloudflare Tunnel for public access.

The architecture was designed during Milestone 4 and provided as part of the implementation specification.

8. Deployment Workflow Designed

The following workflow was defined as the proposed deployment process:



This workflow represents the deployment sequence defined for implementation, rather than a record of implementation work personally performed by me.

9. Technical Decisions and Rationale

9.1 Docker

Decision: Docker was selected for containerization.

Reason: The frontend and backend have separate requirements, and containerization provides an isolated and reproducible deployment environment.

9.2 Docker Compose

Decision: Docker Compose was selected for multi-service management.

Reason: It provides a straightforward mechanism for managing the frontend and backend together.

9.3 Separate Frontend and Backend Containers

Decision: Frontend and backend were planned as separate containers.

Reason:

- Clear service separation.
- Independent dependencies.
- Easier maintenance.
- Better modularity.

9.4 Nginx

Decision: Nginx was selected for the frontend.

Reason:

- Production React build serving.
- Static asset serving.
- Client-side routing support.

9.5 Hugging Face Hub

Decision: Hugging Face Hub was selected for PatchCore model hosting.

Reason: The model checkpoints are approximately 3.75 GB, making external storage more suitable than embedding the model into the repository or Docker image.

9.6 Environment Variables

Decision: API URLs and deployment-specific configuration were planned to be managed through environment variables.

Reason: The API endpoint may differ between local, containerized, and publicly exposed environments.

9.7 Cloudflare Tunnel

Decision: Cloudflare Tunnel was selected for public exposure.

Reason: It provides public accessibility without requiring traditional cloud hosting for the demonstration deployment.

10. Deployment Plan Handoff

After analyzing the requirements and finalizing the deployment methodology, the complete deployment plan was prepared for implementation.

The handoff to **Himabindhu Ravuri** included:

Architecture

- Separate frontend container.
- Separate backend container.
- Docker Compose-based orchestration.
- Nginx for frontend serving.
- External PatchCore model hosting.
- Cloudflare Tunnel for public exposure.

Technology Stack

- Docker.
- Docker Compose.
- Nginx.
- Hugging Face Hub.
- Cloudflare Tunnel.
- Environment variables.
- Git/GitHub.

Configuration

- Frontend API configuration through environment variables.
- Frontend port: 3000.
- Backend port: 5000.

Model Strategy

- PatchCore checkpoints hosted externally.

- Automatic model retrieval proposed during backend startup.

Deployment Workflow

Build → Compose → Model Fetch → Serve → Tunnel → Access → Validate

These details were provided as the implementation specification for the deployment work.

11. Milestone 4 Outcome

The main outcome of Milestone 4 was a well-defined and implementation-ready deployment plan for VisionInspect AI.

The milestone established:

- Deployment requirements.
- Deployment methodology.
- Technology stack.
- Proposed architecture.
- Containerization approach.
- Model-hosting strategy.
- Configuration strategy.
- Public-access strategy.
- End-to-end deployment workflow.
- Technical rationale.
- Implementation requirements.

Most importantly, the deployment methodology was converted into a clear technical specification and **handed over to Himabindhu Ravuri for implementation**.

12. Conclusion

Milestone 4 focused on defining a suitable deployment approach for VisionInspect AI. The deployment requirements were analyzed, the technology stack and architecture were finalized, and the deployment methodology was documented.

The finalized plan was handed over to **Himabindhu Ravuri for implementation**. Thus, my contribution in this milestone primarily involved **deployment planning, technical decision-making, architecture design, and documentation**.